

LeanSearch v2: 面向 Lean 4 定理证明的全局前提检索

Guoxiong Gao^{1,2} Zeming Sun^{1,3} Jiedong Jiang⁴ Yutong Wang^{1,2}
Jingda Xu² Peihao Wu² Bryan Dai^{2,†} Bin Dong^{5,6,7,8,†}

¹ 北京大学数学科学学院

² IQuest Research

³ 京都大学数理解析研究所

⁴ 西湖大学西湖高等研究院

⁵ 北京大学北京国际数学研究中心与新基石科学实验室

⁶ 北京大学机器学习研究中心

⁷ 大湾区大学大湾区高等研究院智能计算中心

⁸ 中关村学院

[†] 通讯作者: cbdai@iquestlab.com, dongbin@math.pku.edu.cn

摘要

在 Lean 4 中证明定理, 往往需要识别一组分散在库中的引理; 这些引理共同使用时, 才能给出简洁证明。我们称这一任务为全局前提检索。现有工具处理的是相邻问题: 语义搜索引擎寻找与查询匹配的单个声明, 而前提选择系统按每个 tactic 步预测有用引理。二者都不能恢复一个完整定理所需的全部前提集合。本文提出 LeanSearch v2, 这是面向该任务的双模式检索系统。其标准模式使用层级非形式化的 Mathlib 语料, 并采用 embedding-reranker 流水线; 在不进行领域特定微调的情况下, 它取得了最先进的单查询检索性能 (nDCG@10 为 0.62, 下一最佳系统为 0.53)。其推理模式以标准模式作为检索底座, 通过迭代的 sketch-retrieve-reflect 循环面向全局前提检索。在一个包含 69 个研究级 Mathlib 定理查询的基准上, 推理模式能在 10 个候选内恢复 46.1% 的真实前提组, 超过强推理检索系统 (38.0%) 以及前提选择基线 (9.3%)。在固定 prover loop 的受控下游评估中, 用 LeanSearch v2 替换其他检索器得到最高证明成功率 (20%, 下一最佳系统为 16%, 无检索为 4%), 说明检索质量会传导到证明生成。我们已开源所有代码、数据和基准。

12

1 引言

Lean 4 及其核心库 Mathlib (The mathlib Community, 2020) 的快速进展, 使形式化在数学共同体中越来越被接受; 该库现在已经包含超过十万个经过验证的声明, 横跨代数、分析、拓扑和数论。AI 系统进一步加速了这一趋势, 已经展现出不断增长的形式化证明生成能力 (Hubert et al., 2025; Ren et al., 2025; Achim et al., 2025)。在这个生态中工作的一个核心挑战, 是有效利用 Mathlib 已经包含的内容。定位正确的库引理有两个互补收益: 一个选得好的结果能把冗长推导压缩成一次 tactic 调用, 而复用既有结果也是共同体明确

¹代码与数据见 <https://github.com/frenzymath/LeanSearch-v2>。

²标准模式已公开提供 API 访问: <https://leansearch.net/>。

的代码规范。Mathlib 优先追求一般性，并主动反对代码重复 (Mathlib Community, 2024)；事实上，“能够找到你所需要的库定理，是形式化中的重要组成部分” (Avigad and Massot, 2020)。这一规范对自动 prover 有直接影响：如果没有有效检索，AI 系统就不得不为每个遇到的问题反复证明库中已经存在的局部版本引理，浪费计算并增加证明难度。

然而，找到正确引理并不容易，主要原因并非库规模很大，虽然规模确实加剧了问题，而是这些联系需要数学推理。有效应用一个强定理，经常要求把证明状态重塑成匹配该定理接口的形式，或者把当前目标泛化、变换成暂时更复杂的形态，只为通过某个看似无关领域的结果走出一条总体上更简单的证明路线。相关引理与问题之间的联系并不依赖共享词汇，而依赖证明策略的逻辑结构。恢复一个完整证明应当依托的支撑引理集合，正是我们称为全局前提检索的任务：识别哪些库结果对定理的整体证明策略是必要的，考虑的是整个证明的需求，而不是任何单一中间状态如何被简化。

示例：全局前提检索

问题。证明对每个素数 p ， $\frac{x^p-1}{x-1}$ 在 $\mathbb{Z}[x]$ 中不可约。

支撑前提 (来自 3 个相距较远的 Mathlib 模块)：

引理	陈述 / 作用
<code>geom_sum_mul</code> Algebra.Ring.GeomSum	$(\sum_{i=0}^{n-1} x^i)(x-1) = x^n - 1$ 建立代数恒等式。
<code>Polynomial.cyclotomic_prime</code> RingTheory.Polynomial.Cyclotomic.Basic	$\Phi_p(x) = \sum_{i=0}^{p-1} x^i$ 将该商识别为第 p 个分圆多项式。
<code>Polynomial.cyclotomic_irreducible</code> RingTheory.Polynomial.Cyclotomic.Roots	对 $n > 0$ ， Φ_n 在 $\mathbb{Z}$ 上不可约。 一步完成证明。

注。问题陈述与这些前提没有共享词汇 (“cyclotomic” “geometric sum” “ Φ ” 都未出现)。识别它们需要关于证明策略的推理，而不是词法匹配；这三个引理串联起来才能闭合证明，缺少任何一个都会让论证不完整。

图 1：一个需要来自相距较远的 Mathlib 模块的三个引理的全局前提检索实例。

图 1 说明了这一任务为什么困难。 $(x^p-1)/(x-1)$ 不可约性的经典证明会使用替换和 Eisenstein 判别法，但在 Lean 中，更高效的证明完全绕过了重新推导，而是借助分圆多项式的既有库结果；一旦建立了代数等价性，就可以一步解决目标。第一个挑战是意识到存在这样的结果，因为问题陈述与分圆多项式机制并无共享词汇。第二个挑战是精确指出三个具体引理，它们必须串联起来才能闭合证明；缺少任何一个都会让论证不完整。在研究生水平及以上的数学中，这种模式极为常见：证明依赖少量分散在库中的支撑结果，它们并非由共同术语连接，而是由证明策略的逻辑结构连接。

目前有两条活跃研究线在为数学构建检索工具，但二者都没有处理全局前提检索。前提选择路线 (Yang et al., 2023; Tao et al., 2025; Shen et al., 2025; Zhu et al., 2025) 针对给定证明状态检索前提，改进 tactic 级证明，但它以逐状态粒度运行，并不考虑整个证明需要什么。推理检索路线由 BRIGHT 基准 (Su et al., 2024) 推动，通过推理感知 encoder、多阶段流水线和基于 LLM 的重排序 (Shao et al., 2025; Long et al., 2025b; Yao et al., 2025)，推进需要多步推断的查询检索。然而，当前基准独立评估每个被检索文档的相关性；这些系统尚未处理这样的场景：来自不同数学领域的多个前提必须共同支撑一个证明，也没有处理形式语言情境中为了匹配库定理接口而有意重塑问题陈述的情况。二者都不是为了恢复整个定理的逻辑一致的前提集合而设计的；在这种集合中，各元素由证明策略的内部逻辑连接，而不仅仅是对同一查询共同相关。

我们提出 **LeanSearch v2**，这是为该任务设计的检索系统。它的标准模式作为基础：构建一个层级非形式化的 Mathlib 语料，并使用两阶段 embedding-reranker 流水线进行检索，不需要领域特定微调。它的推理模式以标准模式作为检索底座，直接面向全局前提检索：给定一个定理陈述，它生成证明草图，经标准

模式为每个证明步骤检索候选项，过滤并判断结果，再迭代修订草图，直到出现一个一致的支撑引理集合。我们在两个新基准上评估：**MathlibQR** (Mathlib Queries)，一个包含 200 个声明、946 条查询的标准模式搜索评估；以及 **MathlibMPR** (Mathlib Merged Pull Requests)，一个包含 69 个定理、带专家标注前提组（每个定理 1 到 8 组）的全局前提检索基准。在 MathlibQR 上，标准模式取得 $nDCG@10 = 0.62$ ，下一最佳系统为 0.53。在 MathlibMPR 上，推理模式能在 10 个候选内恢复 46.1% 的真实前提组，超过推理检索器 (INF-X (Yao et al., 2025)、ReasonIR (Shao et al., 2025)、DIVER (Long et al., 2025b)) 和前提选择基线 (ReProver (Yang et al., 2023)、LeanStateSearch (Tao et al., 2025)、LeanPremise (Zhu et al., 2025))。受控下游实验进一步确认，在保持 prover loop 固定时，这一检索优势会传导到证明成功率。

虽然我们评估了把检索器接入 prover 系统后的下游影响 (§4.3)，但本文有意把自身定位在检索研究层，而不是证明生成层。强 agent 系统 (Chen et al., 2025; Liu et al., 2026; Achim et al., 2025; Ju et al., 2026) 已经表明，能力强的 agent 可以通过反复改写查询来补偿较弱检索器，甚至完全绕过检索、局部证明所需引理。然而，要缩小“仅仅能编译的证明”与高效、惯用证明之间的差距，仍然依赖于识别并使用已有结果；这正是使全局前提检索本身成为有意义任务的核心挑战。

贡献。

- 我们形式化提出全局前提检索：恢复整个定理证明所需的、逻辑一致的库引理集合。我们构建了两个专家策划的 Lean 4 基准：用于单查询搜索的 MathlibQR (200 个声明、946 条查询)，以及用于集合值前提检索的 MathlibMPR (69 个带结构化前提组标注和替代证明路线的定理)。
- 我们提出 LeanSearch v2。它的标准模式在无需领域特定微调的情况下实现最先进的 Mathlib 搜索；它的推理模式以标准模式为底座，通过迭代的 sketch-retrieve-reflect 循环直接面向全局前提检索，在我们的基准上超过推理密集型检索器和前提选择基线。受控下游实验进一步确认，检索质量会传导到端到端证明成功率。
- 据我们所知，这是第一项把推理检索技术带入 Lean 前提检索问题的的工作。

2 相关工作

2.1 推理检索

近期基准开始形式化那些需要超越词法匹配的推理检索任务 (Welleck et al., 2021; Su et al., 2024; Ju and Dong, 2025; Killingback and Zamani, 2025; Abdallah et al., 2026)。Welleck et al. (2021) 在非形式文本中提出了数学参考文献检索。随后，BRIGHT (Su et al., 2024) 定义了推理密集型检索，揭示了标准稠密检索器在面对需要多步推断的查询时会出现显著准确率下降。后续方法通过推理感知 encoder (Shao et al., 2025; Das et al., 2025)、多阶段流水线 (Long et al., 2025b) 和基于 LLM 的重排序 (Weller et al., 2025; Long et al., 2025a) 来处理这一问题。这些基准和方法面向网页文档、科学论文等非形式文本语料，并且独立评估每个被检索文档的相关性。

2.2 Mathlib 上的检索

语义搜索引擎。 若干工具支持在 Lean 4 的 Mathlib 上进行自然语言搜索。Moog³ 和 Loog⁴ 分别提供语义搜索与句法搜索。LeanSearch (Gao et al., 2024a) 将现成 embedding 应用于形式-非形式双语语料。

³<https://www.moog.le.ai>

⁴<https://loog.le-lean-lang.org/>

LeanExplore (Asher, 2025) 结合稠密、稀疏和图信号。Lean Finder (Lu et al., 2025) 在合成用户意图查询上微调。每个系统都为单个自然语言查询返回一个排序列表。

前提选择。 另一条路线检索的是以局部证明状态为条件的前提。ReProver (Yang et al., 2023) 在 Lean 中引入了状态-前提对上的稠密检索。后续工作通过改进负样本挖掘 (Tao et al., 2025)、集成进端到端 prover (Shen et al., 2025)、自动推理后端 (Zhu et al., 2025) 和图增强 embedding (Petrovčič et al., 2025) 来细化这一范式。这些系统专注于寻找能推进当前局部证明状态的前提。

3 方法

LeanSearch v2 以两种模式运行。标准模式 (§3.1) 从自然语言查询中检索单个 Mathlib 声明。它既是独立搜索引擎，也是第二种模式的检索底座。推理模式 (§3.2) 面向全局前提检索：给定形式定理陈述，它迭代生成证明策略草图并查询标准模式，组装简洁证明所需的库前提集合。

3.1 标准模式

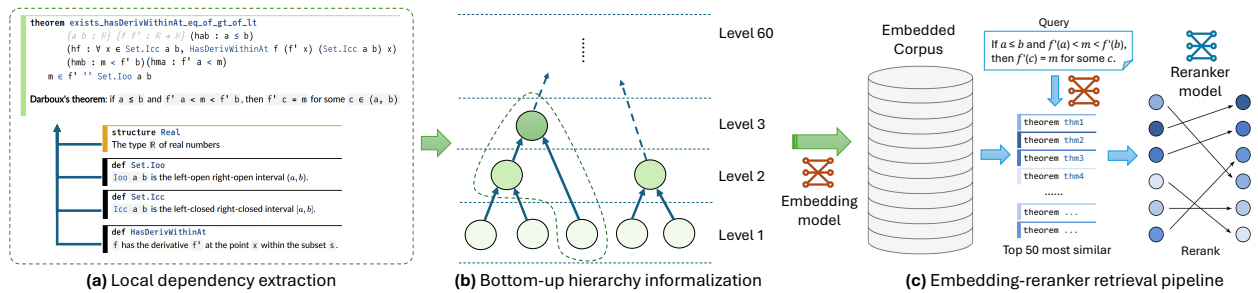


图 2: 标准模式流水线。(a) Jixia 从 Mathlib 中抽取每个声明及其依赖。(b) 声明自底向上非形式化，使每个描述都能利用已经非形式化的依赖。(c) 查询和语料项被嵌入，用于余弦相似度检索；reranker 再细化顶部候选。

标准模式既作为独立搜索引擎，也作为推理模式的检索底座 (图 2)。它包含两个组成部分：非形式化的 Mathlib 语料，以及 embedding-reranker 检索流水线。

非形式化语料准备。 检索语料覆盖 Mathlib v4.28.0-rc1 中所有可抽取声明 (定义、定理、类型类、实例和其他命名常量)，每个声明都配有对其数学意义的简洁自然语言描述。

我们使用 Jixia⁵ 抽取声明；Jixia 是一个公开可用、开源、非侵入式的 Lean 4 静态分析工具 (图 2a)。它的 Declaration 插件提供源代码层面的元数据 (种类、参数、返回类型、源码范围)；Symbol 插件记录 elaborated constants、它们的类型以及声明间引用图。为提升抽取覆盖率，本文扩展了 Jixia，使其能够：(i) 抽取隐藏的编译器生成声明，例如由结构产生的辅助定义 (图 3)；(ii) 从 elaborated terms 中恢复声明，而不只依赖表面源码；(iii) 支持自定义 Lean 运行时选项，以确保抽取表示与语料构建时的配置一致。

引用图在声明上诱导出一个有向无环图 (DAG)。我们对其进行拓扑排序，并自底向上进行非形式化 (图 2b)：每次调用都会把目标依赖项已经非形式化的描述作为上下文，使描述扎根于数学概念，而不是不透明的形式标识符；已有研究表明，这种依赖感知的非形式化策略，比扁平、无上下文的非形式化能产生更扎实的描述 (Gao et al., 2024b)。非形式化使用 Qwen3-32B (Yang et al., 2025) 和任务特定 prompt；失败项用

⁵<https://github.com/frenzymath/jixia>

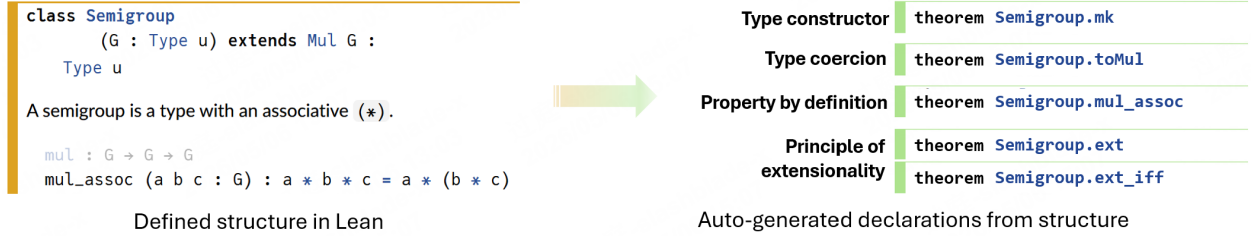


图 3: Mathlib 中隐藏的自动生成声明。

Gemini 2.5 Pro (Comanici et al., 2025) 作为后备重新处理。每个语料项保存声明种类、完全限定名、类型签名、值（如有）、源码位置、依赖以及非形式化描述。

Embedding 与 reranker 流水线。 每个语料项都用 Qwen3-Embedding-8B (Zhang et al., 2025) 编码为固定维度向量。Qwen3-Embedding-8B 是通用 embedding 模型，在文本 embedding 质量标准基准 MTEB (Muenighoff et al., 2023) 上属于最强模型之一，且未进行领域特定微调（图 2c）。对每个条目，我们使用调优后的模板构造结构化 passage：把任务指令与格式良好的形式信息和非形式信息结合起来，包括声明种类、类型签名、非形式化描述，以及对定义、类和实例额外加入 `value` 字段；该字段携带关键语义（不同于定理的 proof term）。随后将这些 passage 嵌入并建立索引，用于余弦相似度检索。

前 50 个候选由 Qwen3-Reranker-8B (Zhang et al., 2025) 重排。它通过预测二元相关性 token，并用 $P(\text{yes})$ 作为分数，来给每个查询-候选对评分。Reranker 接收种类感知的指令，并对 definition 类声明做特殊处理，目标是补偿先前方法 (Gao et al., 2024a) 在定义检索上的困难；那些方法没有区分声明种类。附录 D.1 给出了该展示设计对定义检索影响的消融。重排后的列表构成标准模式的最终输出。

我们不做任务特定训练：embedder 和 reranker 都没有在 Mathlib 数据上微调。领域知识通过非形式化和种类感知 prompting 进入系统，使系统在 Mathlib 演化时更易维护，也更容易迁移到其他快速增长、风格和约定可能不同的 AI 生成形式库。

3.2 推理模式

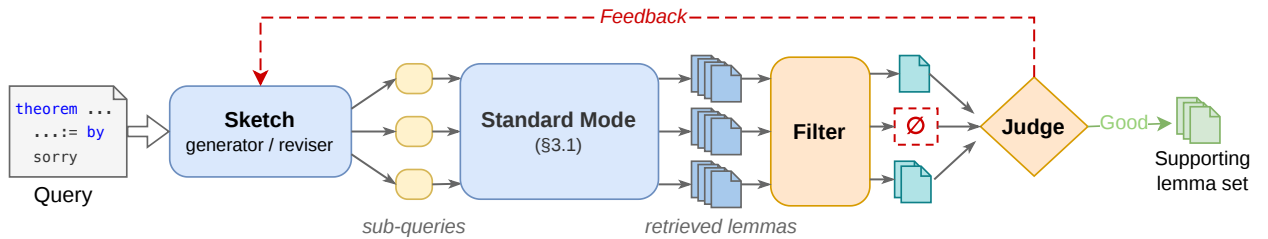


图 4: 推理模式流水线。Sketch generator 把目标定理分解为若干子查询。每个子查询被送入标准模式 (§3.1)，filter 对检索候选进行审查，并可能返回空集 ($\emptyset$)。Judge 检查所有过滤后的输出，要么接受支撑引理集合，要么把结构化反馈（红色虚线箭头）送回 sketch reviser 进入下一轮。

推理模式通过建立在标准模式之上的迭代 sketch-retrieve-reflect 循环来面向全局前提检索（图 4）。给定目标定理后，该循环通过 sketch generator 把目标分解成子查询，经标准模式 (§3.1) 为每个子查询检索候选，对结果进行过滤，再把过滤后的映射提交给可行性 judge；judge 要么接受支撑引理集合，要么把结构化反馈送回 sketch reviser 进入下一轮。标准模式被视作黑盒检索服务：推理模式只向它发出自然语言查询并消费排序结果，绝不深入 embedder 或 reranker。这样的分离使循环能够围绕检索信号进行推理，包括某个查询没有有用结果这一信号，而不是围绕 embedding 或分数推理，也让两种模式可以独立改进。我们使

用 Claude Sonnet 4.5 (Anthropic, 2025) 进行草图生成、judge 和修订, 使用 Kimi K2 Instruct (Team et al., 2025) 做 filter。

草图生成与修订。 给定目标定理 (其形式陈述和非形式描述), sketch generator 会提出分步骤的证明提纲 (图 4 左侧)。每个步骤包含数学操作的自然语言描述、所依赖的辅助上下文, 以及按标准模式预期风格表述的检索查询。我们不要求 generator 写 Lean 代码或命名具体 Mathlib 引理; 它的任务是在“哪些结果应当存在以及每个结果如何贡献”的粒度上承诺一条证明策略, 把引理识别交给检索底座。Sketch reviser 使用相同接口, 但额外消费此前被拒绝的草图和 judge 反馈。

文档过滤器。 对当前草图中的每个子查询, 标准模式返回一个候选声明排序列表 (图 4 中部)。Filter LLM 会把每个候选的元数据 (种类、类型签名、值、非形式化陈述) 与查询放在一起审查, 并独立标记其相关或不相关。关键在于, filter 可以返回空集: 当没有候选真正有助于计划中的步骤时, 一个也不保留。这个空信号使 judge 能区分“检索器为此步骤找到了支持”和“检索器在这里没有找到任何有用内容”, 而这两种情况需要不同修订; 简单的 top- k 规则会混淆它们。

可行性 judge 与反思循环。 收集完所有逐步骤过滤输出后, judge 检查完整的过滤映射, 并决定整个草图是否能由库支撑 (图 4 右侧)。判断是二元的: 如果可支撑, 循环终止, 并把逐查询过滤列表用 nDCG 风格 rank discount 汇总, 形成推理模式的最终输出 (公式与去重规则见附录 B.1); 如果不可支撑, judge 会产生结构化反馈, 指出哪些步骤失败以及原因 (例如缺少前提、候选暗示了不同证明路线、或类型约束不可满足), 然后把控制权交给 sketch reviser, 使修订草图重新进入 retrieve-filter-judge 流水线。我们把循环限制为初始草图之后最多三轮修订; 如果到那时 judge 仍未接受, 就采用最新的过滤输出。附录 D.3 给出了与无反思版本循环的消融比较。循环迭代统计见附录 C.1。

4 实验

我们在三个任务上评估 LeanSearch v2。搜索 (§4.1) 在经典 Mathlib 搜索设定中评估标准模式 (§3.1): 给定自然语言查询, 检索匹配声明, 并用 nDCG@ k 和 Recall@ k 评分。全局前提检索 (§4.2) 在 §1 形式化的任务上评估推理模式 (§3.2): 给定定理陈述, 检索支撑引理集合。基准构建和真实标注详见 §4.2。证明 (§4.3) 从外部衡量检索质量: 把每个检索器接入固定证明生成工作流 (*simple reflection loop*, 见 §4.3), 并在 FATE-H (Jiang et al., 2025) 和 MathlibMPR-Prop 上报告端到端证明成功率。FATE-H 是 FATE 代数系列中的研究生水平层级 (FATE-M / H / X 横跨本科到博士后难度), 含 100 个问题; MathlibMPR-Prop 是 MathlibMPR (§4.2) 的 50 题子集。第一个任务把标准模式与现有 Mathlib 搜索引擎比较; 后两个任务把推理模式与推理密集型检索器和 Lean 侧前提选择系统比较。据我们所知, 先前没有评估把这两个家族放在同一 Lean 基准上。

4.1 搜索: 查询到声明的检索

搜索任务衡量标准模式作为 Mathlib 搜索引擎的质量: 给定自然语言查询, 恢复单个匹配声明。我们在 MathlibQR (Mathlib Queries) 上评估。该基准通过从所有主要数学主题和所有主要 Mathlib 声明种类中采样 200 个 Mathlib 声明构造而成; 每个声明最多配有六种查询风格 (Lean 风格、LaTeX、普通英文、概念 slogan、非正式 nickname、特殊实例), 总计 946 条查询行。真实声明根据数学冷僻程度和技术复杂性标注为 *Easy* 或 *Hard* (评分细则见附录 A.1); 每条查询继承其声明的标签。该基准有意覆盖多样查询风格和声明种类, 使系统不能通过过拟合单一风格而成功; 更多例子和完整策划细节见附录 A.1。

表 1: MathlibQR 上的搜索任务。数值是在共享 171 声明语料子集（四个系统 Mathlib 快照的交集）上、810 条查询行的 $nDCG@k$ 和 $Recall@k$ 。LLM judge 报告 Claude Sonnet 4.5 在每个系统 2430 次 blind judgments 上的平均名次 ($\downarrow$ 越好, 范围 1–4)。每列最佳值加粗。

System	nDCG@ k			Recall@ k			LLM judge
	1	5	10	10	50	100	mean rank
LeanExplore	0.246	0.358	0.393	0.569	0.743	0.789	3.32
LeanFinder	0.370	0.514	0.533	0.698	0.824	0.875	2.87
LeanSearch v2 (retriever-only)	0.340	0.472	0.494	0.657	0.790	0.830	2.18
LeanSearch v2 (rerank)	0.470	0.601	0.623	0.780	0.847	0.858	1.63

我们比较四个系统：LeanExplore (Asher, 2025), 一个混合稠密/稀疏/图引擎；LeanFinder (Lu et al., 2025), 一个带学习 reranker 的微调稠密检索器；完整 retrieve-rerank 流水线的 LeanSearch v2；以及去掉 Qwen3-Reranker-8B 阶段的 LeanSearch v2 retriever-only 消融。由于这些系统面向不同 Mathlib 快照（即某个声明可能存在于一个系统而不存在于另一个系统），主报告使用 MathlibQR 的公平子集，即所有系统快照中都存在真实声明的 810 条查询行，从而控制快照漂移而非检索质量；更宽松子集 (937 行和完整 946 行) 结果见附录 C.4。

我们报告二元单文档相关性下的 $nDCG@k$ 和 $Recall@k$ ，并额外采用遵循 Asher (2025) 的 LLM-as-judge 协议：把每个系统的前 5 个结果以三次独立随机排列呈现给 Claude Sonnet 4.5, judge 对四个系统给出严格总排序 1–4。我们报告每个系统在 2430 次 blinded judgments 上的平均名次（越低越好）；位置偏差和跨轮一致性诊断（包括 primacy-bias 校正）见附录 B.2。

LeanSearch v2（表 1 最后一行）在多数指标上领先。LLM judge 列印证了这一排序，LeanSearch v2 的平均名次为 1.63。附录 C.2 和附录 D.1 报告分片结果和逐步消融：v2-rerank 几乎在每个分片单元都领先，rerank 步骤在 $k \leq 10$ 时贡献约 +10 个 $nDCG/Recall$ 点，种类感知 prompting 又增加 +2–3 个点。

4.2 全局前提检索

全局前提检索任务在 §1 的表述上评估推理模式 (§3.2)：给定定理陈述，恢复其证明所需的全部库引理集合。我们构建 **MathlibMPR** (Mathlib Merged Pull Requests)，这是一个来自已合并 Mathlib pull requests 的 69 个定理基准。对每个 PR，我们抽取主陈述、目标的非形式描述，以及其 tactic 证明实际调用的前提集合。真实标签被组织成前提组（每个查询 1 到 8 组，平均 2.96 组），组内引理对给定证明步骤而言可以互换；对部分查询，形式化专家还把等价证明重路由标注为替代路线。该基准的核心设计选择是数学实质性：候选 PR 经过过滤，只保留那些不是平凡重述、特例或记号改写的定理，因此即便是单组查询，也体现非平凡的前提使用。与传统推理检索基准相比，全局前提检索难度明显更高，且呈现不同特征；详细比较见附录 A.2.3。完整策划细节和例子见附录 A.2。

我们比较两个互补基线家族。第一类是推理密集型检索路线 (INF-X-Retriever (Yao et al., 2025)、ReasonIR (Shao et al., 2025)、DIVER (Long et al., 2025b))，我们把它们运行在层级非形式化的 Mathlib 语料 (§3.1) 上，使其读取的每个声明都具有与 LeanSearch v2 相同的依赖感知自然语言形式。INF-X 和 DIVER 公开的所有消融都逐项报告，以便比较包含每个系统最强配置。第二类是 Lean 侧前提选择路线，它以目标定理的形式证明状态为条件（通过 Lean 4 REPL 抽取；见附录 B.3）：ReProver (Yang et al., 2023)、Lean-Premise (Zhu et al., 2025) 和 LeanStateSearch (Tao et al., 2025)。推理检索器和我们的推理模式流水线以非形式描述 + 形式陈述为输入；前提选择系统以形式证明状态为输入，匹配其公开查询契约。逐系统输入细节见附录 B.3。

表 2: MathlibMPR 上的全局前提检索任务。数值为百分比。*Recall (group)*: top- k 命中的真实前提组比例, 对查询宏平均。*Covered*: 若某条完整证明路线 (原始路线或专家标注替代路线) 的所有前提组都被 top- k 命中, 则为 1。上半块: 消费非形式描述 + 形式陈述的推理检索器; 子行给出公开消融。下半块: 消费抽取证明状态的前提选择系统。每列最佳值加粗。

System	Recall@ k (group)					Covered@ k				
	5	10	20	30	50	5	10	20	30	50
<i>Reasoning retrieval (informal + formal query)</i>										
INF-X-Retriever	21.2	28.2	33.9	40.0	42.6	15.9	18.8	21.7	24.6	24.6
+ query rewriter	13.2	17.4	25.2	28.4	38.3	10.1	10.1	17.4	17.4	24.6
ReasonIR	23.9	26.9	33.3	35.6	43.8	17.4	18.8	20.3	21.7	30.4
DIVER (full pipeline)	27.7	38.0	46.6	52.8	55.0	18.8	24.6	29.0	33.3	37.7
retriever only	21.6	24.7	40.6	45.2	48.5	15.9	15.9	26.1	31.9	33.3
retriever + query expansion	22.2	27.6	40.8	43.0	47.9	15.9	17.4	26.1	27.5	31.9
retriever + GroupRank	29.7	38.5	46.2	52.8	55.0	18.8	24.6	27.5	33.3	37.7
LeanSearch v2 (reasoning)	36.7	46.1	50.8	56.4	57.1	27.5	30.4	34.8	43.5	43.5
<i>Premise selection (proof-state query)</i>										
ReProver	3.0	5.5	7.6	8.8	10.3	0.0	1.4	2.9	4.3	4.3
LeanPremise	4.1	4.8	8.0	9.9	11.5	1.4	1.4	4.3	4.3	5.8
LeanStateSearch	6.5	9.3	12.5	16.0	20.0	2.9	2.9	7.2	10.1	13.0

我们在 $k \in \{5, 10, 20, 30, 50\}$ 上报告两个 top- k 指标。**Recall (group)** 是被命中的真实前提组比例 (如果组内任一可互换引理出现在 top- k 中, 则该组计为命中), 对查询宏平均。**Covered** 是逐查询指示量: 某条完整证明路线 (原 PR 路线或专家标注替代路线) 是否每个前提组都被命中。只使用原 PR 路线作为成功标准的消融见附录 C.3。

LeanSearch v2 (reasoning) 在表 2 的每个单元上领先: 相对于最强推理基线 DIVER full pipeline, v2 在 group-recall@10 上提升 +8.1 个点 (46.1 vs. 38.0), 在 Covered@10 上提升 +5.8 个点 (30.4 vs. 24.6)。这一优势反映了两个设计选择: 推理模式在整体证明策略的粒度上规划, 而不是只规划下一个 tactic 步, 因此子查询更匹配检索意图; judge-reviser 循环会在语料无法支撑计划前提时重路由草稿, 从而过滤掉原则上有效但实际检索不到的路线。在推理检索块中, INF-X query rewriter 反而弱于裸 INF-X retriever (group-recall@10 为 17.4 vs. 28.2): 该 rewriter 训练目标是把冗长、推理密集查询蒸馏成短搜索短语, 但我们的查询本身已经是简洁形式陈述加非形式描述, 因此蒸馏没有太多有用工作可做, 还容易引入转述漂移。我们还观察到, DIVER full pipeline 在推理检索块中最接近我们的推理模式结果; 这与其多阶段设计一致, 因为它在 query expansion、retrieval 和 listwise reranking 上投入了大量 inference-time LLM 能力, 使比较处在双方都投入相近推理资源的制度下。前提选择块在所有 k 上恢复的前提组比例都较低; 我们将其理解为作用域不匹配, 而不是系统质量差距: 前提选择系统专注于寻找推进局部证明状态的前提, 而全局前提检索的集合值目标超出其预期作用域。我们把它们纳入比较, 是因为它们是 Lean 侧检索的主导路线。

4.3 证明: 下游证明生成证据

证明任务测试 §4.2 中观察到的检索质量排序是否会传导到端到端证明生成。我们采用固定证明生成 workflow, 即 *simple reflection loop*: 单个 prover LLM 尝试目标; 每次编译失败时, 工作流把 verifier trace 送入 retrieve-and-reflect 步骤, 然后进行下一次尝试。所有配置中 prover 和 reflection LLM 都是 Claude Sonnet 4.5, 运行 8 轮反思; 何时计为 solved 的标准详见附录 B.4。我们在 FATE-H (Jiang et al., 2025)

表 3: 证明任务: 在固定 simple reflection loop 下改变检索器, 观察端到端证明成功率。 *#solved* 统计工作流在 8 轮反思内生成 Lean 验证通过、无 sorry 证明的问题数量。FATE-H (Jiang et al., 2025) 含 100 题; MathlibMPR-Prop 是 MathlibMPR 中形式陈述允许统一 tactic-mode `:= by sorry` 格式的 50 题子集 (附录 A.2)。每列最佳值加粗。

Retriever	FATE-H ($n=100$)	MathlibMPR-Prop ($n=50$)
no retrieval	4 (4.0%)	2 (4.0%)
LeanFinder	12 (12.0%)	5 (10.0%)
LeanStateSearch	7 (7.0%)	3 (6.0%)
INF-X-Retriever	16 (16.0%)	5 (10.0%)
LeanSearch v2 (standard mode)	14 (14.0%)	5 (10.0%)
LeanSearch v2 (reasoning mode)	20 (20.0%)	7 (14.0%)

(跨子领域采样的 100 个代数问题) 以及 *MathlibMPR-Prop* (50 个问题) 上评估。MathlibMPR-Prop 是 MathlibMPR 中形式陈述允许统一 tactic-mode 格式 `:= by sorry` 的子集 (附录 A.2), 覆盖的数学主题范围与父基准相同。

我们比较六种配置: (i) **无检索** (prover 只看到目标); (ii) **LeanFinder**; (iii) **LeanSearch v2 标准模式**; (iv) **LeanStateSearch**; (v) **INF-X-Retriever**; 以及 (vi) **LeanSearch v2 推理模式**。每个检索器都使用与其公开契约匹配的输入格式: Lean 侧语义搜索引擎 (LeanFinder、LeanSearch v2 标准模式) 接收当前目标的自然语言改写; LeanStateSearch 接收从 Lean 4 REPL 中 prover 当前尝试首次错误处抽取的形式证明状态; INF-X-Retriever 和 LeanSearch v2 推理模式接收定理的非形式描述 + 形式陈述。语义搜索引擎只在 reflection 期间触发, 因为其查询依赖 prover 当前尝试; 以定理陈述为条件的检索器在初始尝试和 reflection 期间都会触发。完整输入处理见附录 B.4。

LeanSearch v2 推理模式 (表 3 最后一行) 在两个数据集上都取得最高成功率 (FATE-H 上 20.0%, MathlibMPR-Prop 上 14.0%), 而且四个以定理陈述为条件的检索器之间的相对排序, 与表 2 中 MathlibMPR group-recall 排序一致。两个 Lean 侧语义搜索引擎只在 reflection 时查询, 表现介于无检索基线和能在一开始看到完整定理陈述的检索器之间。这完成了我们的评估: 候选集合层面的检索质量会传导到端到端证明成功率, 而不会被 prover 和 reflection loop 抵消; 在各数据集上下游相对排序与上游检索排序一致。

5 局限性

第一, 我们只在 Lean 4/Mathlib 上评估。该方法在设计上与语言无关, 但能否经验迁移到 Coq、Isabelle 等其他证明器, 以及它们不同的库约定, 仍有待展示。第二, 我们的下游实验通过一个极简 reflection loop 工作流隔离检索质量, 该工作流一次只证明一个定理。系统在高能力 agent 系统 (Chen et al., 2025; Achim et al., 2025; Ju et al., 2026) 中的行为尚未探索; 在 repository-level formalization 等复杂任务中, 检索会与规划和 tactic 生成交互。第三, 本文聚焦证明构造中的检索。其他检索密集型 Lean 任务, 例如定位规范定义, 或在构造陈述时浏览类型类层级, 可能需要不同的查询表述和评估标准, 本文没有研究。

6 结论

本文形式化提出了全局前提检索, 并介绍了 LeanSearch v2。它是一个分阶段系统, 标准模式在无需领域特定微调的情况下实现最先进的 Mathlib 搜索; 推理模式通过迭代 sketch-retrieve-reflect 循环面向集合值前提检索。两个专家策划基准 MathlibQR 和 MathlibMPR 提供了评估基础; 据我们所知, 这是第一项把推理检索技术带入 Lean 前提检索问题的研究。推理模式在 MathlibMPR 上领先所有基线, 受控下游实验确认

这一优势会传导到证明成功率。我们希望本文提出的任务表述、基准和系统，有助于把检索确立为形式数学进展中的一个独立轴线。

致谢

本工作部分得到中国国家重点研发计划项目 2024YFA1014000、教育部基础学科与交叉学科突破计划 (JYB2025XDXM113) 以及新基石研究员项目的支持。

参考文献

- Abdelrahman Abdallah, Mohamed Darwish Mounis, Mahmoud Abdalla, Mahmoud SalahEldin Kasem, Mostafa Farouk Senussi, Mohamed Mahmoud, Mohammed Ali, Adam Jatowt, and Hyun-Soo Kang. Mm-bright: A multi-task multimodal benchmark for reasoning-intensive retrieval. *arXiv preprint arXiv:2601.09562*, 2026.
- Tudor Achim, Alex Best, Alberto Bietti, Kevin Der, Mathis Fédérico, Sergei Gukov, Daniel Halpern-Leistner, Kirsten Henningsgard, Yury Kudryashov, Alexander Meiburg, et al. Aristotle: Imo-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.
- Anthropic. System card: Claude Sonnet 4.5. <https://www.anthropic.com/claude-sonnet-4-5-system-card>, 2025.
- Justin Asher. Leanexplore: A search engine for lean 4 declarations. *arXiv preprint arXiv:2506.11085*, 2025.
- Jeremy Avigad and Patrick Massot. Mathematics in lean, 2020.
- Jiangjie Chen, Wenxiang Chen, Jiacheng Du, Jinyi Hu, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Wenlei Shi, et al. Seed-prover 1.5: Mastering undergraduate-level theorem proving via learning from experience. *arXiv preprint arXiv:2512.17260*, 2025.
- Gheorghe Comanici, Eric Bieber, Mike Schaeckermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.
- Debrup Das, Sam O’ Nuallain, and Razieh Rahimi. Rader: Reasoning-aware dense retrieval models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 19981–20008, 2025.
- Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. A semantic search engine for mathlib4. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8001–8013, 2024a.
- Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated lean 4 dataset. *arXiv preprint arXiv:2410.10878*, 2024b.

- Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2025. doi: 10.1038/s41586-025-09833-y.
- Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Yongle Hu, Jingting Wang, Nailin Guan, Peihao Wu, Chunbo Dai, Liang Xiao, et al. Fate: A formal benchmark series for frontier algebra of multiple difficulty levels. *arXiv preprint arXiv:2511.02872*, 2025.
- Haocheng Ju and Bin Dong. Mirb: Mathematical information retrieval benchmark. *arXiv preprint arXiv:2505.15585*, 2025.
- Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, et al. Automated conjecture resolution with formal verification. *arXiv preprint arXiv:2604.03789*, 2026.
- Julian Killingback and Hamed Zamani. Benchmarking information retrieval models on complex retrieval tasks. *arXiv preprint arXiv:2509.07253*, 2025.
- Junqi Liu, Zihao Zhou, Zekai Zhu, Marco Dos Santos, Weikun He, Jiawei Liu, Ran Wang, Yunzhou Xie, Junqiao Zhao, Qiufeng Wang, et al. Numina-lean-agent: An open and general agentic reasoning system for formal mathematics. *arXiv preprint arXiv:2601.14027*, 2026.
- Meixiu Long, Duolin Sun, Dan Yang, Yihan Jiao, Lei Liu, Jiahai Wang, BinBin Hu, Yue Shen, Jie Feng, Zhehao Tan, et al. Grouprank: A groupwise paradigm for effective and efficient passage reranking with llms. *arXiv:2511.11653*, 2025a.
- Meixiu Long, Duolin Sun, Dan Yang, Junjie Wang, Yecheng Luo, Yue Shen, Jian Wang, Hualei Zhou, Chunxiao Guo, Peng Wei, et al. Diver: A multi-stage approach for reasoning-intensive information retrieval. *arXiv preprint arXiv:2508.07995*, 2025b.
- Jialin Lu, Kye Emond, Kaiyu Yang, Swarat Chaudhuri, Weiran Sun, and Wuyang Chen. Lean finder: Semantic search for mathlib that understands user intents. *arXiv preprint arXiv:2510.15940*, 2025.
- Mathlib Community. Mathlib contribution guidelines. <https://leanprover-community.github.io/contribute/>, 2024.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037, 2023.
- Job Petrovčič, David Eliecer Narvaez Denis, and Ljupčo Todorovski. Combining textual and structural information for premise selection in lean. *arXiv preprint arXiv:2510.23637*, 2025.
- Auguste Poiroux, Viktor Kuncak, and Antoine Bosselut. Leaninteract: A python interface for lean 4. <https://github.com/augustepoiroux/LeanInteract>, 2025.
- ZZ Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanxia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

- Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen-tau Yih, Pang Wei Koh, et al. Reasonir: Training retrievers for reasoning tasks. *arXiv preprint arXiv:2504.20595*, 2025.
- Ziju Shen, Naohao Huang, Fanyi Yang, Yutong Wang, Guoxiong Gao, Tianyi Xu, Jiedong Jiang, Wanyi He, Pu Yang, Mengzhou Sun, et al. Real-prover: Retrieval augmented lean prover for mathematical reasoning. *arXiv preprint arXiv:2505.20613*, 2025.
- Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han-yu Wang, Haisu Liu, Quan Shi, Zachary S Siegel, Michael Tang, et al. Bright: A realistic and challenging benchmark for reasoning-intensive retrieval. *arXiv preprint arXiv:2407.12883*, 2024.
- Yicheng Tao, Haotian Liu, Shanwen Wang, and Hongteng Xu. Learning an effective premise retrieval model for efficient mathematical formalization. *arXiv preprint arXiv:2501.13959*, 2025.
- Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.
- Sean Welleck, Jiacheng Liu, Ronan Le Bras, Hannaneh Hajishirzi, Yejin Choi, and Kyunghyun Cho. Naturalproofs: Mathematical theorem proving in natural language. *arXiv preprint arXiv:2104.01112*, 2021.
- Orion Weller, Kathryn Ricci, Eugene Yang, Andrew Yates, Dawn Lawrie, and Benjamin Van Durme. Rank1: Test-time compute for reranking in information retrieval. *arXiv preprint arXiv:2502.18418*, 2025.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Kaiyu Yang, Aidan Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36:21573–21612, 2023.
- Yichen Yao, Jiahe Wan, Yuxin Hong, Mengna Zhang, Junhan Yang, Zhouyu Jiang, Qing Xu, Kuan Lu, Yinghui Xu, Wei Chu, Emma Wang, and Yuan Qi. Inf-x-retriever: A pragmatic framework for reasoning-intensive dense retrieval, 2025. URL <https://github.com/yaoyichen/INF-X-Retriever>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025.

A 基准策划

本节给出基准策划方法、关键特征和示例，帮助读者更清楚地理解基准与实验结果。我们介绍两个基准：面向经典形式定理搜索任务的 **MathlibQR**，以及面向全局前提检索的 **MathlibMPR**。详细描述如下：MathlibQR 见附录 A.1，MathlibMPR 见附录 A.2。

A.1 搜索 Mathlib 定理：MathlibQR

MathlibQR 面向经典定理检索任务而设计：给定用户查询，目标是选出最相关的形式定理（文档）。

A.1.1 策划与特征

该基准包含 200 个文档和 946 个查询；多个查询可以对应同一个文档。这样的设计反映了人类或 AI 生成搜索查询的多样性。每个文档最多可以关联六种不同查询风格，即 Lean-centric、LaTeX 表达式、纯自然语言、slogan、nickname 和 special instances，每种风格最多一个查询。这种多样性专门捕捉 Lean 用户的异质性：他们对 Lean 声明和抽象的熟悉程度不同；数学家的成熟程度也不同，可能用自然语言、LaTeX 公式、slogan 或 nickname 表达查询。

最后一类 *special instances* 处理一个常见但此前研究不足的挑战：用户常常心里有具体应用场景，却不知道 Mathlib 中对应陈述的精确形式。例如，用户可能希望应用一个关于 $\mathbb{R}^n$ 中凸集的定理，却不知道 Mathlib 中有更一般的版本。在这种情况下，替代 $\mathbb{R}^n$ 的真实条件包含 `[Semiring k]` `[PartialOrder k]` `[AddCommMonoid E]` `[SMul k E]`，需要专家形式化知识才能预见。设计良好的形式定理搜索引擎，即便在这种苛刻情形下也应返回相关结果。不同查询风格的其他例子见附录 A.1.2。

覆盖范围方面，该基准横跨广泛数学领域，包括代数、分析、逻辑、几何、概率、数论、拓扑和范畴论，并覆盖 Mathlib 的所有主要声明类型，例如定理、定义和实例。每个文档还根据两个维度被分类为 easy 或 hard：数学冷僻程度和技术复杂性。

为构建这样一个有广泛代表性的基准，我们首先请形式化专家从 25 个顶层 Mathlib 文件夹中各选 8 个文档（总共 31 个，排除 `Control`、`Deprecated`、`Lean`、`Tactic`、`Testing` 和 `Util`），以确保声明类型和难度层级上的平衡覆盖。在最终确定 200 个文档后，专家为每个文档尽可能生成最多六种查询风格。

总计我们收集了 946 个查询（详细统计见表 4），基于构建时最新发布的 Mathlib v4.29.1。由于 Mathlib 持续演化，LeanFinder、LeanExplore 和本文工作的语料构建于不同版本。因此，有一个包含 171 个文档、810 个查询的子集落在所有被比较搜索工具共享的共同语料中（第 4.1 节）。

表 4: MathlibQR 200 个条目的基准统计。

(a) 按查询类型

	Lean	LaTeX	Natural	Slogan	Nickname	Special Case
Query Number	199	200	199	197	128	23

(b) 按难度

	Easy	Hard
Document Number	101	99

A.1.2 示例

这些示例展示了同一个数学结果如何通过多样查询风格表达，反映了从形式化专家到数学家等不同用户搜索同一定理的不同方式。基准中每个文档都配有四到六种这样的风格，以捕捉搜索意图的异质性。

示例 1. `Finset.all_card_le_biUnion_card_iff_existsInjective'`. Hall 婚姻定理是组合学经典结果：在 n 个男性和 n 个女性之间存在完美匹配，当且仅当任意 k 个男性共同喜欢至少 k 个女性。这类组合定理经常出现在应用导向语境中，因此我们把一个具体特殊情形纳入查询。

1. <code>Finset.all_card_le_biUnion_card_iff_existsInjective'</code>	
Lean	$(\forall s : \text{Finset } \iota, \#s \leq \#(s.\text{biUnion } t)) \iff \exists f : \iota \rightarrow \alpha, \text{Injective } f \wedge \forall x, f\ x \in t\ x$
LaTeX	$\forall S, S \leq \bigcup_{i \in S} t(i) \iff \exists f \text{ injective}, f(i) \in t(i)$
Natural	一个有限系统存在 distinct representatives 系统，当且仅当每个子族覆盖了足够多的元素。
Slogan	Hall 条件刻画 SDR 的存在性。
Nickname	Hall's marriage theorem
Special case	n 个男性和 n 个女性之间存在完美匹配，当且仅当每个 k 元男性子集共同喜欢至少 k 个女性。

示例 2. `exists_continuous_zero_one_of_isClosed`. Urysohn 引理表明，对正规拓扑空间 X 中任意两个不交闭集 S 和 T ，存在连续函数 $f : X \rightarrow \mathbb{R}$ ，使得 $f|_S = 0$ 、 $f|_T = 1$ ，并且对所有 $x \in X$ 有 $0 \leq f(x) \leq 1$ 。该定理既以名称广为人知，也以 slogan 广为人知。我们还纳入了把它应用到实数上的特殊情形；对这种查询风格而言，恢复一般正规空间表述并不平凡。

2. <code>exists_continuous_zero_one_of_isClosed</code>	
Lean	<code>for disjoint closed s t in a NormalSpace, exists continuous f with f = 0 on s, f = 1 on t</code>
LaTeX	$\exists f \in C(X, [0, 1]), f _S = 0, f _T = 1 \text{ for } S, T \text{ disjoint closed in normal } X$
Natural	在正规空间中，两个不交闭集可以被一个连续函数分离，该函数在一个集合上取 0，在另一个集合上取 1。
Slogan	正规空间中的不交闭集可由连续函数分离。
Nickname	Urysohn's lemma
Special case	在 $\mathbb{R}$ 中， $[0, 1]$ 和 $[2, 3]$ 可由连续 ramp 分离。

示例 3. `AlexandrovDiscrete`. Alexandrov-discrete space (或 Alexandrov topology) 是一种拓扑空间，其中任意族开集的交仍为开集；这一条件严格弱于离散性。由于该名称不如性质本身熟悉，该定义更自然地通过描述性风格 (Lean、LaTeX、自然语言) 而不是 nickname 搜索。

3. AlexandrovDiscrete

Lean	a TopologicalSpace where $\bigcap_0 S$ is open for any family S of opens
LaTeX	$\bigcap_{i \in I} U_i$ open for arbitrary index set I
Natural	一个拓扑空间，其中任意开集族的交仍然开。
Slogan	任意开集交保持开。
Nickname	Alexandrov-discrete space
Special case	—

示例 4. Condensed.instAB4CondensedMod. 一些陈述被存储为实例而不是定理。Slogan “CondensedMod satisfies AB4” 捕捉了这样一个例子。我们还加入 Lean 查询 `instance : AB4 (CondensedMod R)`，这是形式化专家搜索该结果最直接的方式。

4. Condensed.instAB4CondensedMod

Lean	<code>instance : AB4 (CondensedMod R)</code>
LaTeX	CondensedMod R satisfies Grothendieck axiom AB4
Natural	Condensed modules 满足 Grothendieck AB4 (无限直和是正合的)。
Slogan	CondensedMod satisfies AB4
Nickname	—
Special case	—

A.2 全局前提检索：MathlibMPR

MathlibMPR 是用于全局前提检索的基准：给定一个数学问题，任务是检索代表关键数学步骤的定理；这些定理组合起来能给出解答。名称 MathlibMPR 来自其构建方法：该基准由 Mathlib 中已合并的 pull requests 构成。

A.2.1 策划与特征

MathlibMPR 包含 69 个从已合并 Mathlib pull requests 中抽取的数学问题。每个问题配有自然语言陈述、形式陈述和前提组列表。每个前提组包含若干数学上等价但形式化不同的引理，并标注为 *original* 或 *alternative*。Original 表示这些引理作为 Mathlib 合并证明中的关键步骤出现；alternative 表示这些引理在另一条同样可行的证明中发挥关键作用。

策划过程中，形式化专家选择了晚于我们实验中所有搜索引擎语料的已合并 PR (第 4.2 节)。对每个选中 PR，专家识别主定理，总结关键数学步骤，并标注相应 Mathlib 声明。只要可能，他们还会构造 Mathlib 中可行的替代证明，并标注构成这些证明关键步骤的引理。

所有选中 PR 的合并时间，都比实验中 prover 模型 (Claude Sonnet 4.5, 第 4.3 节) 的知识截止时间晚六个月以上，以确保不存在数据污染。

为保证与 prover pipeline 兼容，我们进一步把问题过滤为 50 个具有统一格式的形式陈述（每个都以 `:= by sorry` 结尾），使所有前提选择工具能正确运行。

A.2.2 示例

本节突出全局前提检索的几个关键性质：浏览 typeclass 继承链、提出中间化约步骤、容纳多条可行证明策略，以及使用抽象机制。其中一些性质也出现在一般推理检索任务中，另一些则是形式数学特有的；附录 A.2.3 会给出详细比较。

示例 1 (中间化约步骤)。 PR 30112 证明 Kallenberg 的 “isolation of randomness” 引理：给定标准 Borel 空间 Y 和 Markov kernel $\kappa : X \rightsquigarrow Y$ ，存在联合可测映射 $f : X \rightarrow I \rightarrow Y$ ，使得对每个 $a : X$ ，单位区间 I 上均匀测度经 $f(a, \cdot)$ 的 pushforward 等于 $\kappa(a, \cdot)$ 。

该证明首先需要化约到 $Y = I$ 的情形，使用的事实是：到标准 Borel 空间的每个 Markov kernel 都可以表示为 $[0, 1]$ 上均匀分布在一族联合可测函数下的 pushforward。没有这一化约，用于构造逆 CDF 的技术引理就无法适用。这个化约步骤高度非平凡：它不会从原陈述中自然显现，需要相当的数学经验，通常也需要多轮推理尝试才能发现。

示例 2 (多种证明策略)。 PR 36490 证明范畴论中的如下陈述：给定伴随 $F \dashv G : \mathcal{C} \rightleftarrows \mathcal{D}$ ，以及 $\mathcal{C}$ 上的 Grothendieck 拓扑 J 和 $\mathcal{D}$ 上的 K ，左伴随 F 从 J 到 K 是 cocontinuous (FU 的每个 K -cover 拉回为 U 的 J -cover)，当且仅当 G 保持从 K 到 J 的 covers (X 的每个 K -cover 都被送为 GX 的 J -cover)。

PR 中合并的证明通过伴随的 unit 和 counit 对 sieves 做初等操作。还有一条概念上更简洁的替代路线： F cocontinuous 当且仅当其 pullback G^* 保持 sheaves，这又当且仅当 G 保持 covers。这一路线更优雅，但需要当前 Mathlib 中尚不可用的大量 sheafification 机制，使其难以在合理证明长度内执行。因此采用了初等方法。

示例 3 (抽象机制)。 PR 31754 证明每个非空 light profinite space 都是 profinite spaces 范畴中的 injective object：对 light profinite 且非空的 S ，任何交换方块

$$\begin{array}{ccc} X & \hookrightarrow & Y \\ g \downarrow & \swarrow & \\ S & & \end{array}$$

其中 $X \hookrightarrow Y$ 是 profinite spaces 范畴中的 monomorphism，都存在 lift $Y \rightarrow S$ 扩张 g 。

证明分两阶段。首先，对有限空间 S 和有限空间之间的映射建立结果。然后，把任意 light profinite space S 写成有限空间的 cofiltered sequential limit，并应用“传递到极限”的抽象机制，把结果转移到一般情形。两个预备步骤都是专门为了喂给这个抽象论证而构造的：如果不能识别问题需要 passage to a cofiltered limit，单个引理本身没有作用。因此，解此题要求能从具体陈述抽象到 cofiltered limits 的框架；这一步关键依赖识别具体问题背后的正确抽象结构的能力。

A.2.3 与推理检索任务比较

上一节突出了全局前提检索特有的几个挑战：提出中间化约步骤、在多种证明策略中选择，以及使用抽象机制。本节将这些特征与传统推理检索基准 BRIGHT (Su et al., 2024) 比较。下面两个例子来自其附录。

示例 1 (Su et al. (2024) 表 27)。 该例来自 LeetCode 问题：

查询：

给定 ‘n’ 个非负整数，表示每个柱宽为 ‘1’ 的海拔图，计算下雨后能接住多少水。

Example 1:

Input: height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]

Output: 6

Explanation: 上述海拔图（黑色部分）由数组 [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] 表示。在该情形中，有 6 个单位的雨水（蓝色部分）被接住。

Example 2:

Input: height = [4, 2, 0, 3, 2, 5]

Output: 9

Constraints:

- $n == \text{height.length}$
- $1 \leq n \leq 2 \times 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

用于寻找文档的思维链推理:

这个问题可以用双指针方法解决，并使用动态规划思想来跟踪每个柱子左右两侧的最大高度。我们可以寻找同样使用这些技术的其他示例代码。

示例 1 分析。这个检索问题也需要检索前推理：必须先回忆可能的解题策略，例如双指针技术、动态规划，然后再搜索相关文档。不过，它缺少全局前提检索中出现的独特挑战，也就是从具体陈述抽象到一般框架，或者基于可用机制在多条可行证明策略中选择。这里的推理更依赖联想和识别，而不是结构抽象，因此在推理深度上低于 Mathlib 中研究级形式前提检索。

示例 2 (Su et al. (2024) 表 29)。 该例来自 2015 AMC 10B 第 15 题：

查询:

Hamlet 镇每匹马对应 3 个人，每头牛对应 4 只羊，每个人对应 3 只鸭。下面哪一个不可能是 Hamlet 镇中人、马、羊、牛和鸭的总数？

(A) 41 (B) 47 (C) 59 (D) 61 (E) 66

用于寻找文档的思维链推理:

我们可以用 Chicken McNugget Theorem 解决这个问题。可以寻找其他同样应用该定理的解法。

示例 2 分析。这个检索问题依赖单个知名定理：一旦解题者识别出该问题是 Chicken McNugget Theorem 的应用，检索目标基本固定。无需寻找化约步骤、评估多条可行证明策略，或把具体陈述提升到抽象框架。因此，该任务所需推理明显低于研究级形式前提检索。

B 实验设置

B.1 跨子查询的推理模式聚合

我们按如下方式给推理模式的最终输出排序。一个推理模式草图会把目标定理分解成多个子查询；每个子查询经过标准模式，再经文档过滤器后处理，得到逐子查询的已接受 Mathlib 声明排序列表。同一个声明

可能出现在多个子查询列表中，而不同子查询之间的原始检索分数不可直接比较，因为每个子查询都在不同指令和查询下形成自己的检索分布。因此，我们丢弃原始分数，只按 rank position 聚合。

具体地，令 $L_1, \dots, L_m$ 为某个被接受草图的逐子查询过滤列表（每个 L_j 是长度为 $|L_j|$ 的 declaration id 列表）。对每个出现在 L_j 中、0-indexed rank 为 $i_j(d)$ 的声明 d ，我们从列表 j 赋予位置贡献 $1/\log_2(i_j(d)+2)$ ，并定义 d 的聚合分数为

$$s(d) = \sum_{j: d \in L_j} \frac{1}{\log_2(i_j(d)+2)}.$$

第 j 个列表会给予其 rank-1 条目贡献 1.00，rank 2 条目贡献 0.63，rank 3 条目贡献 0.50，依此类推；同一声明在出现的各列表中的贡献相加。推理模式最终排序是对并集中的 $\{s(d)\}$ 降序排列，再截断为 top- k 。

我们是在尝试过初始设计后采用该 rank-only 规则的。初始设计按每个声明跨子查询取得的最大原始余弦分数或 rerank 分数聚合；它在小 k 上经验效果很差。原因是 embedder 的余弦相似度和 reranker 的 $P(\text{yes})$ 输出都以逐子查询指令和查询为条件，因此不同子查询上的分数处在不可比较的范围。把原始分数替换为只依赖位置的 discount，完全消除了跨子查询校准问题，并在小 k 上带来一致改进，同时不牺牲 $k \geq 100$ 的性能。

B.2 搜索任务的 LLM-as-judge 协议

表 1 中的 LLM-as-judge 列遵循 Asher (2025) 为 Mathlib 搜索引入的协议，并做两处修改，以适配四系统比较和缓解位置偏差。我们简要说明该协议如何工作，以及运行了哪些自检。

每个被检索声明都以六字段块呈现给 judge：完全限定 Mathlib 名称、声明种类、人类可读非形式名称、形式签名（截断到 800 字符）、值或证明体（截断到 400 字符）、非形式化自然语言陈述（截断到 600 字符）。所有四个系统的结果都从标准模式使用的单一共享元数据源中 hydrate，因此 judge 阅读材料的形式在系统之间完全一致。对 810 个 active queries 中每一个，我们通过确定性逐查询 seed，从四个系统的 24 个可能排列中无放回采样三个不同排列。每个 prompt 中系统匿名呈现为 *System A / B / C / D*，因此 judge 不能利用系统身份；跨轮逐查询排列使每个系统暴露在多个标签位置。Judge 是 Claude Sonnet 4.5，调用参数为 temperature=0、max_tokens=600，并被提示对四个 top-5 结果集输出严格总排序 1-4，格式为 JSON 对象，其 ranking 字段是 ["A", "B", "C", "D"] 的一个排列。Prompt 禁止平局；移除 JSON code fences 后，parser 在每个系统 2430 次判断中未观察到格式错误响应。对每个 (slice, system) 单元，我们报告平均名次（越低越好；范围 1-4），即在 overall slice 上每个系统有 $810 \times 3 = 2430$ 次判断。

两个自检支持把结果解读为稳定。第一是跨轮一致性：三轮中的逐轮平均名次最多只相差几个百分位；round- i 和 round- j 排名向量之间的平均 pairwise Spearman correlation 始终较高。第二是位置偏差检查：judge 确实存在 primacy bias，也就是每个系统在标签位置 A 时平均名次都优于位置 D，摆幅约 0.42-0.58 个名次（表 5）。但排列过程把每个系统近似均匀分配到四个标签位置（在 2430 次判断上每个单元 574-640 次，均匀期望 607.5），因此该偏差会在系统层面平均掉，报告的 overall mean ranks 是去偏后的。

表 5：以系统随机标签位置为条件的平均名次。数值是由逐查询 seed 分配到该位置的查询上的平均值。

System	Position A	Position B	Position C	Position D
LeanSearch v2 (rerank)	1.290	1.642	1.718	1.865
LeanSearch v2 (retriever only)	1.854	2.156	2.327	2.370
LeanFinder	2.522	2.877	3.091	2.985
LeanExplore	3.011	3.379	3.479	3.429

B.3 全局前提检索：逐系统输入和输出处理

表 2 中的八个推理检索行和三个前提选择行，在两个程序方面不同：它们接收什么作为逐查询输入，以及输出在评分前如何后处理。每个家族内部所有系统都产生 Mathlib declaration ids 的 top-100 列表；Mathlib 声明名称归一化后唯一，因此输出处理不是逐系统区别，而是统一 doc-id 匹配。输入则分成三个家族：我们的推理模式流水线、推理检索器、前提选择系统。我们逐一说明。表 6 汇总了逐行流水线。

LeanSearch v2 推理模式。 这里我们把推理模式 (§3.2) 视为黑盒：输入是原始 query 字段，也就是非形式描述和形式 Lean 陈述的拼接；输出是可变长度的 Mathlib declaration ids 排序列表。在 **MathlibMPR** 上，平均输出长度为每个查询 30.6 个文档 ($P_{25} = 21$, median = 26, $P_{75} = 38$, max = 90)；我们在每个 $k \leq 100$ 上给系统评分，但其 $k \geq 50$ 单元几乎与 $k = 30$ 相同，因为系统在大 k 时受输出长度约束，不会填充到固定长度。

推理检索器。 推理检索行以同一原始 query 为输入，并直接送入相应 encoder，除了两个系统会在检索前包裹查询。INF-X-Retriever 的 rewriter regime 会运行 infly/inf-query-aligner，先生成一个短搜索查询，再送入 infly/inf-retriever-v1-pro encoder。DIVER 的 qexpand regime 会运行 DeepSeek-R1-Distill-Qwen-14B 作为 query-expansion reasoner，其输出与原始查询拼接后送入 Diver-Retriever-4B encoder。其他 INF-X、ReasonIR 和 DIVER 配置直接送入原始查询。输出是 encoder (或 rerank 增强 encoder) 在与标准模式相同 Mathlib 语料上的 top-100。

前提选择。 ReProver、lean-premises encoder 和 LeanStateSearch 都消费证明状态 (目标 + 假设上下文)，而不是非形式描述，以匹配其公开查询契约。我们通过给每个查询的形式陈述追加 := sorry 并经 LeanInteract (Poiroux et al., 2025) 重新运行到 Lean 4 REPL，来抽取证明状态；REPL 会发出一个 Sorry record，其中 goal 字段就是目标证明状态。在 **MathlibMPR** 上，69/69 个查询都成功抽取。每个系统从自己偏好的前提语料中检索 top-100。

表 6: MathlibMPR 上的逐系统流水线。 *Input:* I/F = 非形式 + 形式陈述；PS = 抽取证明状态。 *Corpus:* Mathlib = 我们的非形式化 Mathlib 快照；LD = LeanDojo Benchmark premises；LP = LeanPremise (Zhu et al., 2025) 发布的前提语料；LSS = premise-search.com 服务端语料。

System (regime)	Input	Pipeline	Corpus
INF-X retriever-only	I/F	retrieve top-100	Mathlib
INF-X + rewriter	I/F	query-aligner → retrieve top-100	Mathlib
ReasonIR	I/F	retrieve top-100	Mathlib
DIVER retriever	I/F	retrieve top-100	Mathlib
DIVER + qexpand	I/F	R1-Distill-14B expand → retrieve top-100	Mathlib
DIVER + rerank	I/F	retrieve top-100 → GroupRank-32B listwise rerank	Mathlib
DIVER full pipeline	I/F	qexpand → retrieve → rerank, score-blended	Mathlib
LeanSearch v2 reasoning	I/F	sketch (Sonnet) → leansearch retriever → filter (Kimi K2) → judge (Sonnet) → rank aggregate	Mathlib
ReProver	PS	byt5-small encoder → retrieve top-100	LD
LeanPremise	PS	all-distilroberta-v1 ft encoder → retrieve top-100	LP
LeanStateSearch	PS	HTTP API → top-100	LSS

B.4 证明：simple reflection loop 中的逐系统输入处理

表 3 每一行都运行同一个 simple reflection loop：prover LLM 产生候选 Lean 证明，Lean REPL 验证；若编译失败，则 reflection step 用 REPL 反馈 (以及可选检索提示) 重新生成证明。逐行差异存在于两

个位置：检索器看到什么查询，以及检索器在循环哪个步骤触发。我们下面描述每一行，然后在表 7 中汇总。所有行共享相同循环超参数：8 轮 reflection、`prover_max_retries=3`、`verifier_max_retries=3`、`verifier_wait=30` 秒；表 3 每一行中 prover LLM 都是 Claude Sonnet 4.5。一个证明计为 solved，当且仅当它能编译且不含 live `sorry`（检查前会去掉 block 和 line comments）。

无检索。 Prover 第一轮只看到非形式描述和形式陈述；每个 reflection 轮接收此前错误证明和 REPL 错误信息。该配置没有检索器，是衡量在相同 loop 中加入检索贡献的锚点。

LeanSearch v2 (标准模式)。 每个 reflection 轮中，LLM 把 prover 当前尝试改写成自然语言查询，查询标准模式，并把顶部命中拼接进 reflection prompt。Query rewrite 是必要的，因为标准模式设计为消费自然语言搜索短语，而 prover 的中间状态是局部 Lean 证明加编译错误。

LeanFinder。 机制上与 LeanSearch v2 标准模式行相同，只是自然语言查询被发送到 LeanFinder Hugging Face Space 的 `gradio_client/retrieve` 端点，HTML table 响应被解析成规范 hit schema 后插入 reflection prompt。

LeanStateSearch。 每一轮（包括第一轮）循环都通过 Lean REPL 抽取形式证明状态：给当前尝试追加 `:= sorry`，运行它，并从所得 Sorry record 中读取 goal/hypothesis context。该状态字符串被送往 LeanStateSearch HTTP API，返回的前提列表被插入下一次 prover prompt。这里不运行 query-rewrite，因为 LeanStateSearch 本来就设计并训练为直接消费形式证明状态。

INF-X-Retriever。 每一轮（包括第一轮）循环都把原始查询字符串发送给包装 INF-X-Retriever encoder 的本地 FastAPI 服务。第一轮查询是非形式描述和形式陈述的拼接，不使用 INF-X-Retriever 提供的 rewriter（见 §4.2 对 rewriter 检索质量影响的讨论）；reflection 轮查询是当前 Lean 代码和 REPL 错误信息的拼接。服务只返回 doc ids，loop 再用我们的 metadata pickle hydrate，填入 prover prompt 所需的形式签名和非形式化陈述。

LeanSearch v2 (推理模式)。 第一轮中，loop 对非形式描述 + 形式陈述运行推理模式 (§3.2)，产生草图和聚合前提集合；草图文本和聚合前提列表都插入 prover prompt。Reflection 轮中，loop 回退到标准模式 reflection 路径：LLM 把 prover 当前尝试改写成自然语言查询，查询标准模式，并把命中返回给 prover。我们不在每个 reflection 轮重新调用推理模式，因为成本会过高；推理模式输出只计算一次，并作为 prover refinement 的固定草图。

表 7：证明任务各行中的检索器集成。 *Round 1:* 第一次 prover 调用时给检索器的输入。*Reflection:* 编译失败后的后续轮输入。“—” 表示该步骤没有检索调用。

Row	Round-1 input	Reflection input	Notes
no retrieval	—	—	
LeanSearch v2 (standard mode)	—	NL queries	retriever 仅在 reflection 触发
LeanFinder	—	NL queries	gradio_client 到 HF Space
LeanStateSearch	proof state	proof state	每轮抽取 proof state
INF-X-Retriever	informal + formal	code + err	raw query, BRIGHT instruction
LeanSearch v2 (reasoning)	informal + formal	NL queries	reasoning sketch 在第 1 轮后固定

C 更多实验结果

C.1 MathlibMPR 上推理模式循环迭代

我们对 **MathlibMPR** 基准上的推理模式进行 instrumentation，记录每个查询中 judge 接受的是初始草图还是修订草图（如有）、需要多少次修订，以及并行分支是否一致。每个查询以 $\text{budget} = 2$ 个分支并行运行，每个分支修订上限为 3，因此每个分支会发出一个初始草图，以及最多三个修订草图后超时。我们报告两个统计：被接受草图的修订次数，以及两个分支的联合结果。

Judge 至少接受一个分支草图的查询有 46/69 个 (66.7%)；其余 23 个查询 (33.3%) 中，每个分支都达到修订上限，此时两个分支的逐子查询过滤列表按附录 B.1 的 rank-discount 规则合并。在 46 个被接受查询中，获胜分支的修订次数如表 8：近三分之二的被接受查询在无修订的初始草图上即被接受，没有被接受查询需要第三次修订。

表 8: 被接受草图的修订次数。 *Initial* = 仅 sketch generator；*revised once / twice* = 接受前调用 sketch reviser 一次或两次。未被接受的 23 个查询中，每个分支都耗尽上限（修订三次）。

Revision count of accepted sketch	# queries	% of 69
0 (initial sketch accepted)	29	42.0%
1 (one revision before acceptance)	11	15.9%
2 (two revisions before acceptance)	6	8.7%
<i>Unaccepted (no branch accepted within 3 revisions)</i>	23	33.3%

分支层面，单个分支会独立终止为 **good** (judge 接受)、**early_stop** (运行时检测到 sibling 已有 accepted output 满足 budget，于是停止进一步工作) 或 **fail** (耗尽修订上限仍未被接受)。两个分支的联合结果见表 9。在 46 个被接受查询中，branch 0 在 31 个查询上产生获胜草图，branch 1 在 15 个查询上产生；这与 branch 0 先初始化、因此更常在 **early_stop** 触发 sibling 之前到达 judge 的事实一致。

表 9: 联合分支状态分布。 分支顺序为启动顺序；**early_stop** 表示一旦 sibling 产生 accepted sketch，运行时取消该分支。

(Branch 0, Branch 1) outcome	# queries
(good, good)	24
(early_stop, good)	14
(good, early_stop)	6
(good, fail)	1
(fail, good)	1
(fail, fail)	23

合起来看，这两个分布支持循环设计。66.7% 的接受率与 MathlibMPR 上最强推理基线在 $k=100$ 时的 Covered rate 相当，说明 judge 的二元可行性决策，是底层检索底座是否实际浮现正确前提的合理代理。没有被接受查询需要最大修订次数，也说明修订上限设置合适：初始草图之后每多一次修订带来的额外接受数递减 ($29 \rightarrow 11 \rightarrow 6$)，因此第四次修订边际收益可能很小。最后，两个分支很少在最终 verdict 上分歧；忽略 **early_stop** 时， $24 + 23 = 47$ 个查询两个分支以相同状态完成。这支持了把两个并行分支作为低成本方差降低机制而非本质不同探索策略的设计选择。

C.2 搜索任务：分片结果

我们在 **MathlibQR** 公平子集 (810 行) 上按三个维度报告 nDCG@10 和 Recall@10: 查询难度 (Easy / Hard)、真实声明种类 (报告 encoder 使用种类特定 prompt 模板的 kinds, 同时报告 overall 行)、查询风格 (六种风格)。在公平子集上计算这些分片, 使每个分片的基础与表 1 的主报告基础相同。以下表格中, **LE** = LeanExplore, **LF** = LeanFinder, **LSv2(r)** = LeanSearch v2 retriever-only, **LSv2(rr)** = LeanSearch v2 with reranking。

表 10: **MathlibQR** 按难度分片。一半查询为 Easy, 一半为 Hard。

Difficulty	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
Easy	0.410	0.568	0.533	0.666	0.599	0.731	0.722	0.829
Hard	0.373	0.491	0.449	0.571	0.534	0.658	0.580	0.722

表 11: 公平子集上 **MathlibQR** 按真实声明种类分片。报告 encoder 使用 kind-specific prompt template 的 kinds (theorem, def, instance) 及 overall 行, 因此逐 kind 单元是独立比较而非穷尽划分。每行最佳值加粗。

Kind	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
overall	0.393	0.533	0.494	0.623	0.569	0.698	0.657	0.780
theorem	0.351	0.617	0.619	0.765	0.546	0.790	0.849	0.914
def	0.441	0.637	0.657	0.764	0.571	0.790	0.829	0.943
instance	0.390	0.661	0.839	0.846	0.619	0.866	0.907	0.938

表 12: **MathlibQR** 按查询风格分片。q1a Lean 风格表述; q1b LaTeX 陈述; q1c plain English 陈述; q2 短概念 slogan; q3 非正式 nickname; q4 特殊实例。

Query style	nDCG@10				Recall@10			
	LE	LF	LSv2(r)	LSv2(rr)	LE	LF	LSv2(r)	LSv2(rr)
q1a Lean	0.340	0.607	0.504	0.622	0.524	0.735	0.624	0.729
q1b LaTeX	0.383	0.551	0.508	0.634	0.561	0.696	0.655	0.807
q1c plain text	0.449	0.528	0.531	0.667	0.647	0.724	0.694	0.824
q2 slogan	0.367	0.441	0.477	0.573	0.518	0.655	0.655	0.720
q3 nickname	0.510	0.607	0.465	0.641	0.685	0.741	0.685	0.843
q4 special case	0.079	0.218	0.317	0.500	0.217	0.348	0.522	0.783

LeanSearch v2 with reranking 在每个报告的 slice $\times$ metric 单元上领先, 除了 q1a Lean 查询上一个近似平局: LeanFinder 在 Recall@10 上以 0.006 微弱领先。领先幅度在更难分片上扩大: 在 *Hard* 查询上, 带 reranking 的 v2 比下一最佳系统领先 +6.4 Recall@10; 在 *q4 special case* 这个对表面匹配最不友好的分片上, 它相对表中下一最佳系统领先 +26.1 Recall@10。表 11 的 kind-sliced 行是独立比较而非基准划分, 因为并非每种声明种类都适合干净的 per-kind prompt template; 在同三种 kind 上隔离 kind-aware instruction 贡献的消融见附录 D.1。

C.3 全局前提检索：替代成功标准

表 2 的核心数字把如下情形计为 solved：某条专家标注证明路线，无论是原 PR 证明还是 curator 的等价替代路线，其每个前提组都被 top- k 命中。为验证 §4.2 的结论不是由允许替代路线驱动，我们报告一个更严格的 Covered 变体，它只使用原 PR 路线；同时也报告同一组 k 值上的主文变体。表 13 对所有系统报告 $k \in \{5, 10, 20, 30, 50\}$ 下的两列。

表 13: MathlibMPR 上的 Covered 变体 ($n=69$, 百分比)。 *Covered (main only)* 要求原 PR 路线所有组都被命中；*Covered (main $\vee$ alt-all)* 还在整条替代路线所有组被命中时给分，是表 2 报告的变体。每列最佳值加粗。

System	Covered (main only)					Covered (main $\vee$ alt-all)				
	5	10	20	30	50	5	10	20	30	50
<i>Reasoning retrieval (informal + formal query)</i>										
INF-X-Retriever	14.5	17.4	18.8	21.7	21.7	15.9	18.8	21.7	24.6	24.6
+ query rewriter	10.1	10.1	14.5	15.9	23.2	10.1	10.1	17.4	17.4	24.6
ReasonIR	15.9	17.4	18.8	18.8	26.1	17.4	18.8	20.3	21.7	30.4
DIVER (full pipeline)	17.4	21.7	24.6	29.0	31.9	18.8	24.6	29.0	33.3	37.7
retriever only	14.5	14.5	23.2	27.5	27.5	15.9	15.9	26.1	31.9	33.3
+ query expansion only	14.5	15.9	23.2	23.2	26.1	15.9	17.4	26.1	27.5	31.9
+ GroupRank rerank only	17.4	21.7	24.6	29.0	31.9	18.8	24.6	27.5	33.3	37.7
LeanSearch v2 (reasoning)	26.1	29.0	33.3	40.6	40.6	27.5	30.4	34.8	43.5	43.5
<i>Premise selection (proof-state query)</i>										
ReProver	0.0	1.4	2.9	2.9	2.9	0.0	1.4	2.9	4.3	4.3
LeanPremise	1.4	1.4	4.3	4.3	5.8	1.4	1.4	4.3	4.3	5.8
LeanStateSearch	2.9	2.9	7.2	8.7	11.6	2.9	2.9	7.2	10.1	13.0

两个变体之间系统相对排序保持不变：LeanSearch v2 reasoning 每列领先，DIVER full pipeline 仍是最强非 v2 推理基线；任何 k 下最强前提选择系统与 v2 reasoning 在 $k=5$ 下的差距，也在两种标准下保持。对强推理系统而言，Covered (main $\vee$ alt-all) 与 Covered (main only) 的绝对差距约为 1–3 个百分点；对前提选择系统则几乎为零。因此，§4.2 的结论不依赖替代路线给分。

C.4 不同基准视角下的 MathlibQR

主报告使用 MathlibQR 的公平视角：每个系统原则上都能返回真实声明，因为查询行被限制到三个 Mathlib 快照 (LeanFinder、本文系统、LeanExplore) 的交集。这是最干净的比较，但会丢弃 136 条真实声明落在一个或多个竞争者快照之外的行。为检查 §4.1 的结论是否在更宽松比较中仍然成立，我们在两个额外视角下重新评分同一组检索器。*at-least-one* 视角 (937 行) 保留真实声明在三个快照中至少一个存在的每条查询；*full* 视角 (946 行) 是未过滤完整基准，其中包括 9 条真实声明不在三个快照中任何一个里的行，因此四个系统都不可能检索到。

LeanSearch v2 with reranking 在每个 (perspective $\times$ metric) 单元上排名第一。相对第二名的领先在公平视角最大 (nDCG@5 相对 LeanFinder +0.087, Recall@10 相对 LeanFinder +0.082)；在更宽松视角下，第二名在某些指标上从 LeanFinder 变成 LeanExplore，但我们的领先仍保持为 *at-least-one* 视角 nDCG@5 上的 +0.13, *full* 视角上的 +0.13。从 *fair* 转向 *full* 时，各列绝对分数下降 4–6 个点，反映了额外 136 行中某些系统按构造不可达；相对排序不受影响。

表 14: 跨 MathlibQR 视角的 LeanSearch v2 与基线比较。每个单元是在对应视角上的 nDCG@ k 或 Recall@ k 。每个 (视角 $\times$ 列) 单元最佳值加粗。

Perspective (n)	System	nDCG@ k			Recall@ k		
		1	5	10	10	50	100
fair (810)	LeanExplore	0.246	0.358	0.393	0.569	0.743	0.789
	LeanFinder	0.370	0.514	0.533	0.698	0.824	0.875
	LeanSearch v2 (rerank)	0.470	0.601	0.623	0.780	0.847	0.858
at-least-one (937)	LeanExplore	0.238	0.351	0.386	0.564	0.735	0.777
	LeanFinder	0.320	0.445	0.461	0.603	0.712	0.757
	LeanSearch v2 (rerank)	0.446	0.576	0.595	0.747	0.810	0.820
full (946)	LeanExplore	0.236	0.347	0.383	0.558	0.728	0.770
	LeanFinder	0.317	0.440	0.457	0.597	0.705	0.750
	LeanSearch v2 (rerank)	0.442	0.570	0.590	0.740	0.802	0.812

D 消融研究

D.1 搜索任务消融

标准模式由两个设计步骤逐步构建：从普通稠密检索器出发，我们先加入 reranker，再给语料编码和 reranker prompt 加入 kind-aware instruction。两个步骤都旨在带来帮助，但贡献并不对称，并且依赖被检索声明的种类。表 15 的消融在 MathlibQR 公平子集上隔离每一步，把分数按三种声明种类拆开 (theorem、def、instance；这些种类使用 kind-specific prompt template)，并报告 Overall 作为并集。

表 15: MathlibQR 公平子集 (810 行) 上标准模式的渐进消融。no rerank: 仅 Qwen3-Embedding-8B。rerank, no kind: 加 Qwen3-Reranker-8B，但语料编码使用 kind-blind 模板（每条记录都编码为 **theorem**），指令也不区分种类。rerank, +kind (生产版): kind-aware 语料编码和指令。每个 (slice $\times$ 列) 单元最佳值加粗。

Slice (n)	Pipeline	R@1	R@5	R@10	nDCG@5	nDCG@10
Overall (810)	no rerank	0.340	0.586	0.657	0.472	0.494
	rerank, no kind	0.441	0.694	0.774	0.572	0.598
	rerank, +kind	0.470	0.714	0.780	0.601	0.623
theorem (291)	no rerank	0.381	0.766	0.849	0.593	0.619
	rerank, no kind	0.612	0.856	0.914	0.746	0.765
	rerank, +kind	0.612	0.859	0.914	0.748	0.765
def (105)	no rerank	0.495	0.752	0.829	0.633	0.657
	rerank, no kind	0.505	0.867	0.924	0.702	0.720
	rerank, +kind	0.581	0.867	0.943	0.738	0.764
instance (97)	no rerank	0.773	0.887	0.907	0.833	0.839
	rerank, no kind	0.742	0.887	0.907	0.819	0.825
	rerank, +kind	0.773	0.897	0.938	0.833	0.846

两个设计步骤在有提升空间的每个分片上叠加。Overall 分片上，nDCG@5 从 embedding-only baseline

的 0.472 上升到生产配置的 0.601，总提升 +0.129；rerank 步骤贡献约四分之三 (+0.101)，kind 步骤贡献剩余四分之一 (+0.029)。按 kind 分解时贡献模式明显不同。Theorem 查询中，rerank 步骤几乎贡献全部提升，把 nDCG@5 从 0.593 推到 0.746，kind 步骤最多再加 0.002，在 291 个查询的分片上几乎与噪声无异；这与 encoder 默认 prompt template 已经像 theorem 一样相符，因此显式标注 theorem 贡献很小。Def 查询中两个步骤都有实质贡献 (nDCG@5 为 0.633 $\rightarrow$ 0.702 $\rightarrow$ 0.738，rerank 约贡献三分之二，kind 贡献其余三分之一)，这是两个干预叠加最清楚的例子。Instance 查询中，embedding-only baseline 已经达到 0.833 nDCG@5，提升空间很小；rerank-no-kind 配置下降 0.014，kind 步骤又把它拉回 0.833，因此在此分片上 kind 步骤的作用是避免小幅退化，而不是驱动提升。在所有四个分片上，生产版 rerank-with-kind 配置在每个指标上至少不弱于任一更简单变体，且从不更差，因此两步构建得到数据支持。

D.2 使用较弱 prover backbone 的 simple reflection loop

表 3 使用 Claude Sonnet 4.5 作为 prover 运行 simple reflection loop。我们通过把同三种检索模式——无检索 (L0)、LeanSearch v2 标准模式 (L1)、LeanSearch v2 推理模式 (L2 *Sketch* 变体)——在 FATE-H ($n = 100$) 上用 Kimi K2 Instruct 作为 prover 重新运行，来消融该选择。循环中的其他角色 (标准模式 reflection 路径上的 query rewriter、推理模式内部的 sketch generator / filter / judge、REPL verifier) 保持与表 3 对应 Sonnet 行完全一致。

表 16: FATE-H 上的 prover-backbone 消融。每个单元为 100 个问题中 solved 数量 (括号中为百分比)。Sonnet 列复现表 3 的 L0 / L1 / reasoning-mode 行；Kimi 列只把 prover 角色换成 Kimi K2 Instruct。

Retrieval mode (prover input)	Sonnet prover	Kimi prover
No retrieval	4 (4.0%)	1 (1.0%)
LeanSearch v2 (standard mode)	14 (14.0%)	8 (8.0%)
LeanSearch v2 (reasoning mode)	20 (20.0%)	12 (12.0%)

当 prover 从 Sonnet 降级到 Kimi 时，三种检索模式的相对排序仍保持：推理模式 (12) > 标准模式 (8) > 无检索 (1)。因此，更好检索信息的边际贡献，无论是第一轮的 reasoning-mode sketch 和聚合前提，还是 reflection 时经标准模式提供的 LeanSearch hits，都会传导到较弱的 prover backbone。把 Sonnet 换成 Kimi 后，绝对成功率约下降 40% (14 $\rightarrow$ 8、20 $\rightarrow$ 12)，说明 prover 能力和检索能力对端到端证明成功率的贡献大致是相乘的，而不是互相替代。

D.3 推理模式中的反思轮数

§3.2 中的推理模式循环把修订上限设为三轮，并使用可行性 judge 的接受/拒绝决策决定草图是否可供下游使用。为隔离 judge-driven reflection 步骤的贡献，我们在 FATE-H 上重新运行证明任务，把推理模式配置为零次修订：照常生成初始草图，但忽略 judge verdict，不管 judge 是否会接受，都直接把初始草图的过滤前提集合传给 prover。循环中其他组件 (sketch generator、filter、prover、REPL verifier、超参数) 都固定为生产设置。比较两次运行，即可衡量单次尝试预算下 reflection 步骤的边际贡献。

完整 reflection loop 在 FATE-H 上比无反思变体多解决 4 个问题，即 +4 个百分点。这确认了 §3.2 中的定性设计论证：judge 的结构化拒绝信号能让 sketch reviser 修复初始前提集合无法让 prover 完成的草图。该收益与附录 C.1 的观察一致，其中约三分之一被接受查询在 judge 接受前至少需要一次修订；该绝对收益也足够小，不会削弱主表结论——即便禁用 reflection，推理模式仍在 FATE-H 上获胜，相比表 3 中最强非推理检索器 INF-X-Retriever (16.0%) 保持 +2 点领先。因此禁用 reflection 是有意义但有限的损失；我们

表 17: FATE-H 上的 reflection-rounds 消融 ($n=100$)。 $rfl = 0$: 忽略 judge verdict, 直接把初始草图的过滤前提集合传给 prover。 $rfl = 3$: 生产设置, 在 judge 接受或循环回退到最新过滤输出前, sketch reviser 最多调用三次。其他推理模式组件相同。

Reasoning-mode reflection	# solved	success rate
$rfl = 0$ (judge bypassed)	16	16.0%
$rfl = 3$ (production)	20	20.0%

默认开启它, 因为额外成本适中 (附录 C.1 报告每个被接受查询平均一次 revision call)。

E 计算资源与许可证

E.1 计算资源

GPU 计算使用小规模加速器池; LLM 驱动步骤使用 Claude Sonnet 4.5 和 Kimi K2 Instruct hosted APIs。对标准模式 (§4.1), 用 Qwen3-Embedding-8B 嵌入整个 Mathlib 语料在单个加速器上运行, 约六小时内完成; reranking 按需对逐查询 top-50 执行。在下游实验中, 标准模式作为部署在两个加速器上的服务暴露, 每个查询约 0.2 秒返回排序列表。对推理模式 (§4.2), 在单个查询上运行完整 sketch-filter-judge-reviser 循环, 按当前 API 价格约花费 \$1.10 USD。其余计算——运行推理检索基线 (INF-X、ReasonIR、DIVER 及其子配置)、在抽取证明状态上运行前提选择基线, 以及报告附录中的逐分片和消融——总计约三到四个 GPU 小时。

E.2 既有资产许可证

- Mathlib (The mathlib Community, 2020): Apache 2.0 license。本文使用 v4.28.0-rc1 和 v4.29.1。
- Jixia: Apache 2.0 license。 <https://github.com/frenzymath/jixia>。
- Qwen3-32B (Yang et al., 2025): Apache 2.0 license。
- Qwen3-Embedding-8B 和 Qwen3-Reranker-8B (Zhang et al., 2025): Apache 2.0 license。
- Kimi K2 Instruct (Team et al., 2025): modified MIT license。
- Claude Sonnet 4.5 (Anthropic, 2025): 通过 Anthropic API 访问, 遵循其公开使用政策。
- Gemini 2.5 Pro (Comanici et al., 2025): 通过 Google AI API 访问, 遵循其公开服务条款。
- FATE-H (Jiang et al., 2025): MIT License。

所有基线检索系统 (LeanExplore、LeanFinder、ReProver、LeanPremise、LeanStateSearch、INF-X-Retriever、ReasonIR、DIVER) 均在正文中引用, 并通过其公开代码或 hosted APIs 访问。